Bismillah Construction — Full Application Specification

Version 1.0 | Internal Engineering Document

TABLE OF CONTENTS

---

1. PROJECT OVERVIEW

1.1 What We Are Building

A single-user, web-based operations management system for a construction business managing up to 10 active sites (scalable to 50+). The system digitises the client's current manual workflow — pocket diaries, monthly ledger books, Tally manual entries, and WhatsApp-based communication — into a single structured platform accessible from any phone or laptop browser.

1.2 Users

- **Primary User:** Business owner (Bismillah Construction) — single login
- **Future Phase:** Multi-user with role-based access (Phase 2, separate scope)

1.3 Core Principles

- Mobile-first — owner uses the app primarily on phone
- Offline-tolerant UI (graceful degradation, not full offline mode)
- All data owned and exportable by the client
- Zero vendor lock-in — standard open-source stack throughout

- Demo-to-production continuity — the demo shares the same schema and codebase

---

2. SYSTEM ARCHITECTURE

2.1 High-Level Architecture

2.2 Request Lifecycle

```
        |
        ├──► [If PDF needed] → Generate PDF → Upload to R2
        |
        └──► JSON response → Frontend renders update
```

2.3 Data Flow — Key Operations

Marking Attendance:

```
Owner taps "Mark Attendance" → selects site → marks each worker
→ POST /api/attendance → Prisma writes attendance rows
→ Wage calculation triggered → updated balance stored
→ UI refreshes attendance summary
```

Sending Client Invoice:

```
Owner creates invoice → taps "Send"
→ POST /api/invoices/:id/send
→ PDF generated server-side → uploaded to R2
→ WhatsApp API call → message sent to client number
→ Notification log entry created
→ Invoice status updated to "Sent"
→ UI shows green "Sent + WhatsApp notified" badge
```

Vendor Bill Entry:

```
Owner logs vendor payment → POST /api/vendors/:id/transactions
→ Transaction saved to DB
→ WhatsApp API call → vendor receives bill confirmation
→ Log entry created with timestamp
→ Vendor balance updated
```

Project Closure:

```
Owner marks project "Closed"
→ POST /api/projects/:id/close
→ System aggregates: attendance, materials, transactions, extra work
→ Closure report PDF generated → saved to R2
→ Project status updated → history locked (read-only)
→ PDF download link returned to UI
```

---

3. TECH STACK

3.1 Frontend

Layer	Technology	Version	Purpose
Framework	Next.js	14 (App Router)	Full-stack React framework
Language	TypeScript	5.x	Type safety across frontend + API
Styling	Tailwind CSS	3.x	Utility-first, mobile-first responsive design
Components	shadcn/ui	Latest	Accessible, unstyled base components
Charts	Recharts	2.x	Material/tool usage charts, financial graphs
PDF View	@react-pdf/renderer	3.x	Closure reports, invoice PDFs
Forms	React Hook Form + Zod	Latest	Form state + schema validation
State	Zustand	4.x	Lightweight global state (auth, active site)
HTTP	Axios	1.x	API calls with interceptors for auth tokens
Date	date-fns	3.x	Date formatting, attendance calendar logic

3.2 Backend

Layer	Technology	Version	Purpose
Runtime	Node.js	20 LTS	Server runtime
Framework	Next.js API Routes	14	Backend endpoints, same repo as frontend
Language	TypeScript	5.x	Shared types with frontend
Auth	NextAuth.js	4.x	Session management, JWT, single-user config
ORM	Prisma	5.x	Type-safe DB access, migrations

Layer	Technology	Version	Purpose
Validation	Zod	3.x	Request body and param validation
PDF Gen	@react-pdf/renderer (server)	3.x	Server-side PDF generation
Job Queue	None (Phase 1)	—	Scheduled backups handled by cron endpoint

3.3 Database & Storage

Service	Technology	Purpose
Primary DB	PostgreSQL 15	All structured data
ORM	Prisma	Migrations, queries, type generation
File Storage	Cloudflare R2	PDFs, backups, exported CSVs
Backup	Automated cron → R2 dump	Daily compressed DB snapshot

3.4 External Integrations

Service	Provider	Purpose
WhatsApp Notifications	Meta WhatsApp Cloud API	Client invoice + vendor bill alerts
Domain (optional)	Namecheap / GoDaddy	Custom URL for client access
SSL	Auto (Vercel / Cloudflare)	HTTPS, no manual cert management

3.5 Dev Tooling

Tool	Purpose
ESLint + Prettier	Code quality and formatting
Prisma Studio	DB inspection during development
Postman / Hoppscotch	API testing
Git + GitHub	Version control
Vercel Preview Deploys	Branch-based staging URLs

4. MODULE SPECIFICATIONS

4.1 Project & Site Management

Purpose: Central hub for all site operations. Every other module is scoped under a project.

Data Entities: Project, SiteActivity

Key Screens:

- Dashboard — all projects with status badges, quick stats
- Project detail — tabbed view: Overview | Labour | Materials | Finance | Extra Work | Reports
- New project form — name, location, start date, assigned staff
- Daily activity log — date-stamped notes per site

Business Rules:

- A project has one of three statuses: `ACTIVE`, `COMPLETED`, `CLOSED`
- `CLOSED` projects are read-only — no further entries allowed
- A project can have multiple labourers and electricians assigned
- Deleting a project is not allowed — archive only

API Endpoints:

```
GET    /api/projects                → list all projects
POST   /api/projects                → create project
GET    /api/projects/:id            → project detail + summary
PATCH  /api/projects/:id            → update details / status
POST   /api/projects/:id/close      → trigger closure report generation
GET    /api/projects/:id/activity   → site activity log
POST   /api/projects/:id/activity   → add daily activity entry
```

4.2 Labour & Staff Management

Purpose: Track all workers and electricians, assign to sites, mark daily attendance, calculate wages.

Data Entities: Worker, WorkerType (LABOURER / ELECTRICIAN), Attendance

Key Screens:

- Worker list — filterable by type (labourer / electrician)
- Worker detail — profile, current assignment, attendance history, total wages
- Attendance screen — per-site, per-day grid (present / absent / half-day)
- Wage summary — per worker, per project, per month

Business Rules:

- A worker can be assigned to only one site at a time (primary assignment)
- Attendance can only be marked for the current day or backdated up to 7 days
- Wage = Daily Rate × (Full Days + 0.5 × Half Days)
- Workers are not deleted — they are marked inactive

API Endpoints:

```
GET    /api/workers                → list all workers
POST   /api/workers                → add worker
GET    /api/workers/:id            → worker detail
PATCH  /api/workers/:id            → update worker
GET    /api/workers/:id/attendance → attendance history
POST   /api/attendance             → mark attendance (batch per site per
day)
GET    /api/attendance             → query by project, date range, worker
```

4.3 Material & Tool Tracking

Purpose: Track all materials and tools per project with grade classification. Monitor buy/issue/return/balance cycle.

Data Entities: Item (material/tool/paint/cement), ItemGrade, ProjectInventory, InventoryTransaction

Key Screens:

- Item master list — all materials, tools, paints, filterable by grade
- Per-project inventory — items assigned to site with quantities
- Transaction log — buy / issue / return / balance columns
- Top usage report — top 3 materials, cement, tools by quantity and cost

Business Rules:

- Every item has a Grade: `GRADE_A` (High), `GRADE_B` (Medium), `GRADE_C` (Low)
- Transaction types: `BUY`, `ISSUE`, `RETURN`, `ADJUST`
- Balance = Total Bought − Issued + Returned
- Items cannot go negative in balance — validation enforced at API level

- Tool return status tracked separately (`ASSIGNED`, `RETURNED`, `MISSING`)

API Endpoints:

```
GET    /api/items                          → master item list
POST   /api/items                          → add item to master
GET    /api/projects/:id/inventory         → project inventory
POST   /api/projects/:id/inventory         → add item to project
POST   /api/projects/:id/inventory/txn     → log transaction
(buy/issue/return)
GET    /api/reports/top-usage              → top 3 per category
(material/cement/tool)
```

4.4 Financial & Transaction Management

Purpose: Track all money in (client payments) and money out (vendor payments, site expenses). Provide live cash flow view.

Data Entities: Client, ClientPayment, Invoice, Vendor, VendorTransaction, SiteExpense, CashFlowSummary

Key Screens:

- Client ledger — billed vs received vs pending per client/project
- Vendor ledger — purchase history, paid vs outstanding per vendor/shop
- Cash flow page — running total of in vs out, net position
- Saturday view — clients with dues this week
- Invoice detail — line items, status (draft / sent / paid)

Business Rules:

- Client balance = Total Invoiced − Total Received
- Vendor balance = Total Purchased − Total Paid
- Cash position = Total client payments received − Total vendor + expense outflows
- An invoice triggers a WhatsApp notification when status is changed to `SENT`
- A vendor transaction triggers a WhatsApp notification on save (togglable)
- Invoices cannot be deleted — only voided with a reason

API Endpoints:

```
GET    /api/clients                        → client list
POST   /api/clients                        → add client
GET    /api/clients/:id/ledger             → payment history + balance
POST   /api/invoices                       → create invoice
PATCH  /api/invoices/:id                   → update invoice
POST   /api/invoices/:id/send              → mark sent → trigger WhatsApp
GET    /api/vendors                        → vendor list
POST   /api/vendors                        → add vendor
POST   /api/vendors/:id/transactions       → log purchase/payment →
trigger WhatsApp
GET    /api/cashflow                       → cash flow summary (date
range)
GET    /api/expenses                       → site expense list
POST   /api/expenses                       → log site expense
```

4.5 Extra Work Log

Purpose: Capture all work performed outside the original project scope immediately, so no revenue is lost.

Data Entities: ExtraWork

Key Screens:

- Extra work list — filterable by project, billed/unbilled, date
- Add extra work form — site, date, description, amount, billed toggle
- Unbilled summary — quick view of all unbilled extra work across all projects

Business Rules:

- Every extra work entry is linked to a project
- Status: `UNBILLED` → `BILLED` → `COLLECTED`
- Billed entries are linked to an invoice
- Unbilled entries appear as a warning badge on the project dashboard

API Endpoints:

```
GET    /api/extra-work              → list (filter by project, status)
POST   /api/extra-work              → create entry
PATCH  /api/extra-work/:id          → update status / link to invoice
GET    /api/extra-work/unbilled     → all unbilled across projects
```

(Full detail in Section 7 — Integration Walkthrough)

Data Entities: NotificationLog

Business Rules:

- Every sent notification is logged with: recipient, type, message body, timestamp, status (sent/failed)
- Failed notifications are retried once automatically
- User can manually resend from the notification log screen
- Notification types: `CLIENT_INVOICE`, `VENDOR_BILL`

4.7 Reports & Analytics

Purpose: Replace all manual monthly reporting with on-demand, always-current reports.

Report Types:

Report	Scope	Output
Top 3 Material Usage	Per project / all projects	Chart + table
Top 3 Cement Usage	Per project / all projects	Chart + table
Top 3 Tool Usage	Per project / all projects	Chart + table
Site Expense Report	Per project, date range	Table + PDF
Revenue Report	Per project / all projects	Chart + table
Vendor Purchase Report	Per vendor, date range	Table + PDF
Project Closure Report	Per project (on close)	Full PDF
Cash Flow Report	Date range	Chart + table

Project Closure Report — Contents:

```
Header
  - Project name, site location, start date, close date, total duration
(days)

Labour Summary
  - Table: Worker name | Days present | Half days | Total days | Daily rate
| Total wages

Material Summary
  - Table: Item name | Grade | Qty bought | Qty issued | Qty returned |
Unit cost | Total cost

Tool Summary
  - Table: Tool name | Grade | Assigned date | Return status

Financial Summary
  - Total agreed project value
  - Total invoiced to client
  - Total received from client
  - Outstanding balance
  - Total extra work billed
  - Total material + vendor expenses
  - Net project margin (revenue − expenses)

Extra Work Log
  - Table: Date | Description | Amount | Billed? | Invoice ref

Site Activity Timeline
  - Chronological list of all daily activity log entries

Footer
  - Generated date, sign-off note field
```

API Endpoints:

```
GET    /api/reports/top-usage           → top 3 per category
GET    /api/reports/expenses            → site expense report
GET    /api/reports/revenue             → revenue report
GET    /api/reports/cashflow            → cash flow report
GET    /api/reports/vendor              → vendor purchase report
POST   /api/reports/closure/:projectId  → generate + store closure PDF
GET    /api/reports/closure/:projectId  → retrieve closure PDF URL
```

Data Entities: Contact (type: VENDOR / SHOP / ELECTRICIAN)

Key Screens:

- Directory with tabs: Vendors | Shops | Electricians
- Contact detail — profile, transaction/assignment history, WhatsApp log
- Add/edit contact form

API Endpoints:

```
GET    /api/contacts            → list (filter by type)
POST   /api/contacts            → add contact
GET    /api/contacts/:id        → detail + history
PATCH  /api/contacts/:id        → update
```

4.9 Data Backup & Export

Backup Schedule: Daily at 2:00 AM IST via cron endpoint **Backup Format:** Compressed PostgreSQL dump (.sql.gz) **Retention:** 90 rolling days on Cloudflare R2 **Manual Trigger:** Available from admin panel

Export Options:

- Full data export → CSV (per table)
- Any report → PDF (stored on R2, downloadable link)
- Invoice PDFs → stored on R2 permanently

API Endpoints:

```
POST   /api/admin/backup          → trigger manual backup
GET    /api/admin/backups         → list available backups
GET    /api/admin/backup/:id      → download backup file URL
POST   /api/admin/export/csv      → generate full CSV export
```

---

5. DATABASE SCHEMA DESIGN

5.1 Entity Relationship Overview

```
Project ──────────────────────────── many Workers (via ProjectAssignment)
   │                                        │
   ├── many SiteActivities            many Attendance
```

```
    ├── many ProjectInventory
    │         │
    │         └── many InventoryTransactions ── Item
    │
    ├── many Invoices ──────────────── Client
    │         │
    │         └── triggers NotificationLog
    │
    ├── many SiteExpenses
    ├── many ExtraWork
    └── one ClosureReport

Vendor / Shop ── many VendorTransactions ── triggers NotificationLog
Contact (ELECTRICIAN / VENDOR / SHOP) ── linked to Project assignments
```

5.2 Core Tables

sql

```
-- Projects
projects
  id              UUID PK
  name            VARCHAR(255)
  location        VARCHAR(255)
  status          ENUM(ACTIVE, COMPLETED, CLOSED)
  start_date      DATE
  end_date        DATE nullable
  agreed_value    DECIMAL(12,2)
  notes           TEXT
  created_at      TIMESTAMP
  updated_at      TIMESTAMP

-- Workers
workers
  id              UUID PK
  name            VARCHAR(255)
  type            ENUM(LABOURER, ELECTRICIAN)
  phone           VARCHAR(20)
  daily_rate      DECIMAL(8,2)
  is_active       BOOLEAN
  created_at      TIMESTAMP

-- Project Assignments (Worker ⇔ Project)
project_assignments
  id              UUID PK
  project_id      UUID FK → projects
  worker_id       UUID FK → workers
  assigned_date   DATE
  released_date   DATE nullable

-- Attendance
attendance
  id              UUID PK
  project_id      UUID FK → projects
  worker_id       UUID FK → workers
  date            DATE
  status          ENUM(PRESENT, ABSENT, HALF_DAY)
  created_at      TIMESTAMP
  UNIQUE(project_id, worker_id, date)

-- Items (master list)
items
  id              UUID PK
  name            VARCHAR(255)
  type            ENUM(MATERIAL, TOOL, PAINT, CEMENT)
```

```
  grade          ENUM(GRADE_A, GRADE_B, GRADE_C)
  unit           VARCHAR(50)  -- kg, bag, piece, litre
  unit_cost      DECIMAL(10,2)
  created_at     TIMESTAMP

-- Project Inventory
project_inventory
  id             UUID PK
  project_id     UUID FK → projects
  item_id        UUID FK → items
  qty_bought     DECIMAL(10,2)  DEFAULT 0
  qty_issued     DECIMAL(10,2)  DEFAULT 0
  qty_returned   DECIMAL(10,2)  DEFAULT 0
  -- balance = qty_bought - qty_issued + qty_returned (computed)

-- Inventory Transactions
inventory_transactions
  id             UUID PK
  project_id     UUID FK → projects
  item_id        UUID FK → items
  type           ENUM(BUY, ISSUE, RETURN, ADJUST)
  quantity       DECIMAL(10,2)
  unit_cost      DECIMAL(10,2)
  date           DATE
  note           TEXT
  created_at     TIMESTAMP

-- Clients
clients
  id             UUID PK
  name           VARCHAR(255)
  phone          VARCHAR(20)
  address        TEXT
  created_at     TIMESTAMP

-- Invoices
invoices
  id             UUID PK
  project_id     UUID FK → projects
  client_id      UUID FK → clients
  invoice_number VARCHAR(50) UNIQUE
  amount         DECIMAL(12,2)
  status         ENUM(DRAFT, SENT, PAID, VOID)
  issued_date    DATE
  due_date       DATE
  notes          TEXT
  pdf_url        TEXT nullable
```

```
  created_at    TIMESTAMP

-- Client Payments
client_payments
  id            UUID PK
  invoice_id    UUID FK → invoices
  client_id     UUID FK → clients
  amount        DECIMAL(12,2)
  payment_date  DATE
  method        VARCHAR(50)
  note          TEXT
  created_at    TIMESTAMP

-- Contacts (Vendor / Shop / Electrician directory)
contacts
  id            UUID PK
  name          VARCHAR(255)
  type          ENUM(VENDOR, SHOP, ELECTRICIAN)
  phone         VARCHAR(20)
  specialty     VARCHAR(255) nullable
  address       TEXT nullable
  created_at    TIMESTAMP

-- Vendor Transactions
vendor_transactions
  id            UUID PK
  contact_id    UUID FK → contacts
  project_id    UUID FK → projects nullable
  type          ENUM(PURCHASE, PAYMENT)
  amount        DECIMAL(12,2)
  date          DATE
  description   TEXT
  notified      BOOLEAN DEFAULT false
  created_at    TIMESTAMP

-- Site Expenses
site_expenses
  id            UUID PK
  project_id    UUID FK → projects
  category      VARCHAR(100)
  amount        DECIMAL(12,2)
  date          DATE
  description   TEXT
  created_at    TIMESTAMP

-- Extra Work
extra_work
```

```
  id              UUID PK
  project_id      UUID FK → projects
  date            DATE
  description     TEXT
  amount          DECIMAL(12,2)
  status          ENUM(UNBILLED, BILLED, COLLECTED)
  invoice_id      UUID FK → invoices nullable
  created_at      TIMESTAMP

-- Site Activity Log
site_activities
  id              UUID PK
  project_id      UUID FK → projects
  date            DATE
  description     TEXT
  created_at      TIMESTAMP

-- Notification Log
notification_logs
  id              UUID PK
  type            ENUM(CLIENT_INVOICE, VENDOR_BILL)
  recipient_phone VARCHAR(20)
  message_body    TEXT
  status          ENUM(SENT, FAILED, PENDING)
  reference_id    UUID  -- invoice_id or vendor_txn_id
  reference_type  VARCHAR(50)
  sent_at         TIMESTAMP nullable
  created_at      TIMESTAMP

-- Closure Reports
closure_reports
  id              UUID PK
  project_id      UUID FK → projects UNIQUE
  pdf_url         TEXT
  generated_at    TIMESTAMP
  summary_json    JSONB  -- cached aggregated data
```

5.3 Indexes

sql

```sql
-- Performance-critical indexes
CREATE INDEX idx_attendance_project_date ON attendance(project_id, date);
CREATE INDEX idx_attendance_worker ON attendance(worker_id);
CREATE INDEX idx_inventory_txn_project ON inventory_transactions(project_id,
CREATE INDEX idx_invoices_client ON invoices(client_id, status);
CREATE INDEX idx_vendor_txn_contact ON vendor_transactions(contact_id, date);
CREATE INDEX idx_extra_work_project_status ON extra_work(project_id, status);
CREATE INDEX idx_notification_logs_status ON notification_logs(status, create
```

---

6. STORAGE ARCHITECTURE

6.1 Cloudflare R2 Bucket Structure

```
r2-bismillah-construction/
│
├── invoices/
│   └── {year}/{month}/invoice-{invoice_number}.pdf
│
├── closure-reports/
│   └── {project_id}/closure-report-{date}.pdf
│
├── exports/
│   └── {date}/export-{table_name}.csv
│
└── backups/
    └── {year}/{month}/{date}/db-backup-{timestamp}.sql.gz
```

6.2 Storage Size Projections

Bucket	10 Sites / 3 Yrs	50 Sites / 3 Yrs	50 Sites / 10 Yrs
invoices/	~90 MB	~450 MB	~1.5 GB
closure-reports/	~15 MB	~75 MB	~250 MB
exports/	~20 MB	~100 MB	~350 MB
backups/ (90-day rolling)	~50 MB	~120 MB	~120 MB
Total	**~175 MB**	**~745 MB**	**~2.2 GB**

6.3 R2 Pricing

Tier	Storage	Requests	Cost
Free	10 GB	1M reads / 10M writes/month	₹0
Paid	$0.015/GB after 10 GB	$0.36/million reads	Minimal

At 50 sites / 10 years (~2.2 GB): ₹0 — stays within free tier. **At extreme scale (100+ sites):** ~$0.015/GB = negligible.

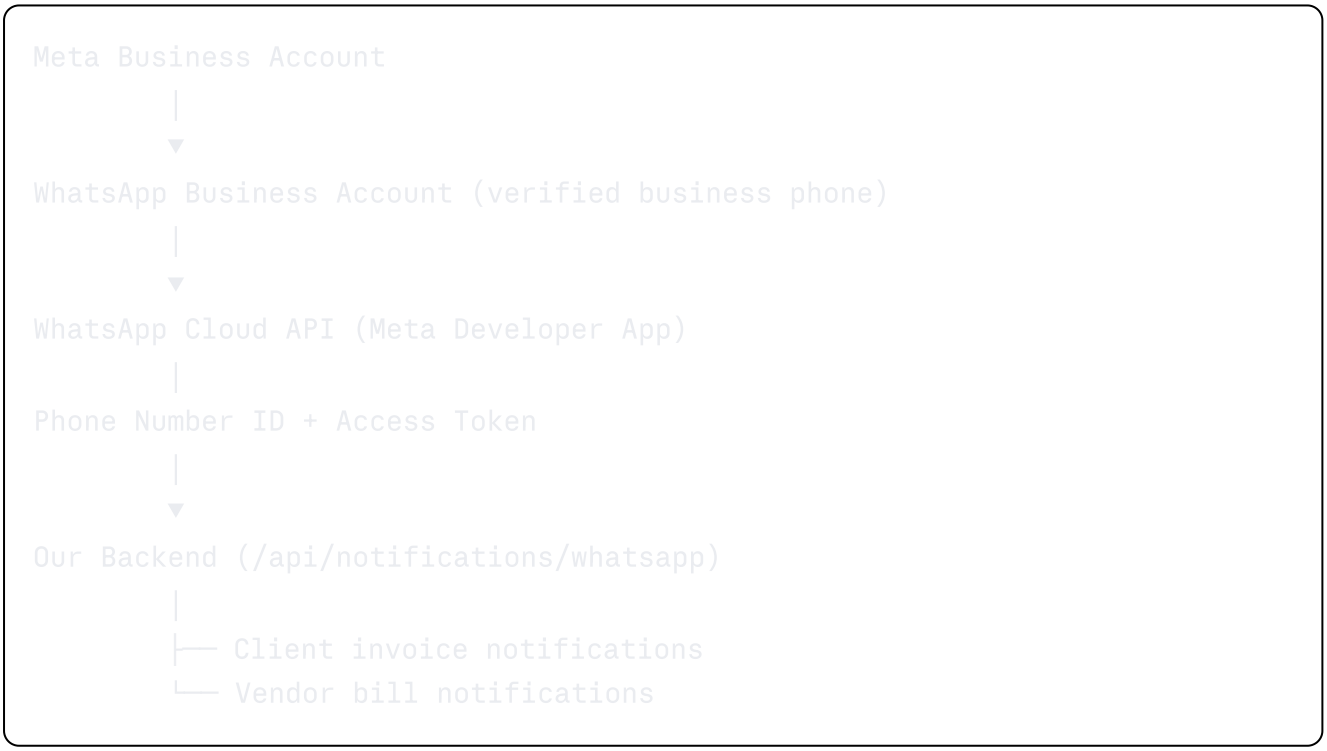
6.4 File Access Pattern

- PDFs are generated server-side, uploaded to R2, and a signed URL is returned to the client
- Signed URLs expire in 1 hour (security — no permanent public links)
- Admin can regenerate links from the backup/report screens

---

7. INTEGRATION WALKTHROUGH — WHATSAPP BUSINESS API

7.1 Setup Overview



7.2 One-Time Setup Steps (Client Does This Once)

1. Create Meta Business Manager account at business.facebook.com
2. Add and verify the business WhatsApp number
3. Create a Meta Developer App → enable WhatsApp product

4. Generate a permanent access token
5. Submit 2 message templates for Meta approval (invoice + vendor bill)
6. Share Phone Number ID and Access Token with us — we configure the environment variables
7. Total time: 2–4 days (Meta approval takes 24–48 hours)

7.3 Message Templates

Template 1 — Client Invoice Notification

```
Template name: client_invoice_sent
Category: UTILITY

Message:
"Hello {{1}}, an invoice has been raised for your project *{{2}}*.

Invoice Number: {{3}}
Amount: ₹{{4}}
Date: {{5}}

Please contact us for payment details. — Bismillah Construction"

Variables:
{{1}} = client name
{{2}} = project name
{{3}} = invoice number
{{4}} = amount
{{5}} = invoice date
```

Template 2 — Vendor Bill Notification

```
Template name: vendor_bill_confirmation
Category: UTILITY

Message:
"Hello {{1}}, this is to confirm a transaction recorded against your
account.

Reference: {{2}}
Description: {{3}}
Amount: ₹{{4}}
Date: {{5}}

— Bismillah Construction"

Variables:
{{1}} = vendor/shop name
{{2}} = transaction reference ID
{{3}} = description
{{4}} = amount
{{5}} = date
```

7.4 API Call Flow (Backend)

```
POST https://graph.facebook.com/v18.0/{PHONE_NUMBER_ID}/messages

Headers:
  Authorization: Bearer {ACCESS_TOKEN}
  Content-Type: application/json

Body (client invoice example):
{
  "messaging_product": "whatsapp",
  "to": "91XXXXXXXXXX",
  "type": "template",
  "template": {
    "name": "client_invoice_sent",
    "language": { "code": "en" },
    "components": [{
      "type": "body",
      "parameters": [
        { "type": "text", "text": "Rajesh" },
        { "type": "text", "text": "Anna Nagar Project" },
        { "type": "text", "text": "INV-2024-047" },
        { "type": "text", "text": "45,000" },
        { "type": "text", "text": "14 June 2025" }
```

```
      ]
    }]
  }
}
```

7.5 Error Handling

- On API failure: notification logged as `FAILED`, user shown a retry button
- Retry once automatically after 5 minutes
- If retry fails: user alerted with option to manually resend
- All attempts (success and failure) logged in `notification_logs` table

7.6 Rate Limits & Costs

Tier	Limit	Cost
Free	1,000 conversations/month	₹0
Paid (Utility)	Unlimited	~₹0.28–₹0.35/conversation
At Bismillah's volume (~70 msg/month)	Well within free	₹0

---

8. API DESIGN OVERVIEW

8.1 Base URL

```
Production:  https://app.bismillahconstruction.in/api
Staging:     https://staging.bismillahconstruction.in/api
Local:       http://localhost:3000/api
```

8.2 Authentication

- All API routes protected by NextAuth session middleware
- Session token in HTTP-only cookie (not localStorage)
- Unauthenticated requests → 401 redirect to /login

8.3 Standard Response Envelope

```
json
```

```
// Success
{
  "success": true,
  "data": { ... },
  "meta": { "page": 1, "total": 42 }
}

// Error
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invoice amount must be greater than 0",
    "field": "amount"
  }
}
```

8.4 Pagination

All list endpoints support:

```
GET /api/projects?page=1&limit=20&sort=created_at&order=desc
```

---

9. AUTHENTICATION & SECURITY

9.1 Auth Flow

```
/login page → email + password form
      |
      ▼
NextAuth credentials provider
      |
      ▼
Single hardcoded user (env vars: ADMIN_EMAIL, ADMIN_PASSWORD_HASH)
      |
      ▼
JWT session → HTTP-only secure cookie
      |
      ▼
All /api/* routes check session → 401 if missing
```

9.2 Security Measures

- Passwords stored as bcrypt hashes (never plaintext)

- HTTPS enforced (auto via Vercel/Cloudflare)
- HTTP-only cookies — no XSS access to session token
- CSRF protection via NextAuth built-ins
- Input validation on all API routes via Zod
- SQL injection prevented via Prisma parameterised queries
- R2 file URLs are signed and time-limited (1 hour)
- WhatsApp access token stored in environment variables — never in code or DB
- Rate limiting on /api/auth/login (5 attempts per 15 min)

9.3 Environment Variables

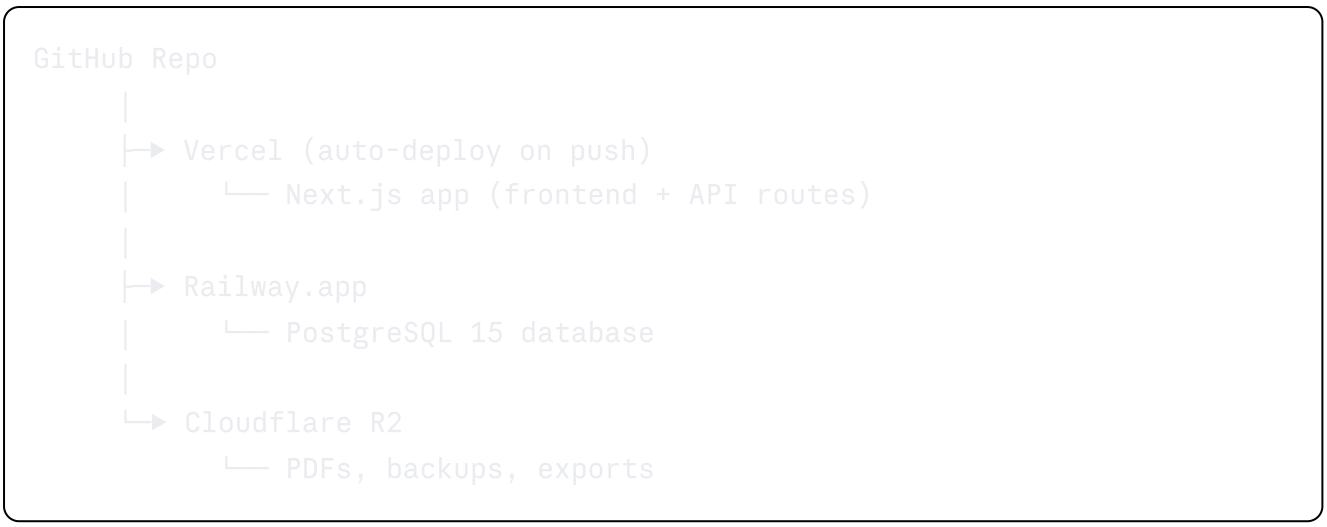
```env
DATABASE_URL=postgresql://...
NEXTAUTH_SECRET=...
NEXTAUTH_URL=https://app.bismillahconstruction.in
ADMIN_EMAIL=owner@bismillah.com
ADMIN_PASSWORD_HASH=bcrypt_hash_here

CLOUDFLARE_R2_ACCOUNT_ID=...
CLOUDFLARE_R2_ACCESS_KEY=...
CLOUDFLARE_R2_SECRET_KEY=...
CLOUDFLARE_R2_BUCKET_NAME=bismillah-construction

WHATSAPP_PHONE_NUMBER_ID=...
WHATSAPP_ACCESS_TOKEN=...

CRON_SECRET=...  # protects the backup endpoint
```

---

10. DEPLOYMENT OPTIONS

Option A — Vercel + Railway (Managed, Recommended)

Architecture:



Pros:

- Zero server management — fully managed platforms
- Auto-scaling handled by Vercel
- Preview deployments on every PR/branch
- SSL auto-provisioned
- Railway includes automated DB backups

Cons:

- Slight vendor dependency (Vercel, Railway)
- Railway free credit can run out if DB queries are heavy
- Less control over server environment

Cost Estimate:

Service	Free Tier	Paid
Vercel	Free (hobby)	$20/month if limits hit
Railway	$5 credit/month (covers DB)	$10–20/month
Cloudflare R2	10 GB free	$0.015/GB after
Total Year 1	**₹0**	**~₹835–₹1,670/month if upgraded**

Setup Time: ~2–3 hours

Option B — Docker on VPS (Self-Hosted, Full Control)

Architecture:

Pros:

- Complete control over the server environment
- Single monthly fixed cost (~₹420–₹500/month for Hetzner CX11)
- No per-service pricing surprises
- Easy to SSH in and debug
- Database and app on same machine — low latency

Cons:

- Requires basic server management knowledge
- Manual SSL renewal setup (Certbot / Let's Encrypt)
- Manual DB backups to R2 (we set this up in a cron job)
- No auto-scaling (not needed at this scale)

Cost Estimate:

Service	Cost
Hetzner CX21 VPS (2 vCPU, 4 GB RAM)	€3.79/month (~₹340/month)
Cloudflare R2	₹0 (free tier)
Domain (optional)	₹1,000/year
Total Monthly	**~₹340–₹430/month**

Setup Time: ~4–6 hours (Docker Compose config, Nginx, SSL, cron)

Option Comparison

Factor	Vercel + Railway (A)	Docker VPS (B)
Setup complexity	Low	Medium

Factor	Vercel + Railway (A)	Docker VPS (B)
Monthly cost (Year 1)	₹0	~₹340
Monthly cost (Year 2+)	₹0–₹1,670	~₹340 (fixed)
Server management needed	No	Minimal
Preview/staging URLs	Auto (Vercel)	Manual
Scaling	Auto	Manual
Best for	Faster launch, less ops	Cost predictability, control
Recommendation	**Default choice**	**If long-term cost matters**

11. DEVELOPMENT ROADMAP

Phase 0 — Foundation (Week 1)

- [ ] Repository setup (Next.js 14, TypeScript, Tailwind, shadcn/ui)
- [ ] Prisma schema defined and initial migration run
- [ ] NextAuth single-user auth configured
- [ ] Deployment pipeline connected (Vercel + Railway or VPS)
- [ ] Cloudflare R2 bucket created and SDK configured
- [ ] Environment variables configured on all environments
- [ ] Base layout — navigation, sidebar, mobile responsiveness shell

Phase 1 — Core Operations (Weeks 2–5)

- [ ] Project/site management module (CRUD, status, dashboard)
- [ ] Worker management (labourer + electrician lists)
- [ ] Site assignment system
- [ ] Attendance marking (per site, per day)
- [ ] Wage calculation
- [ ] Daily site activity log

Phase 2 — Financial Module (Weeks 6–8)

- [ ] Client management + ledger
- [ ] Invoice creation and management
- [ ] Client payment recording
- [ ] Vendor/shop/contact directory
- [ ] Vendor transaction logging
- [ ] Site expense tracking

- [ ] Cash flow page (amount rotation)
- [ ] Saturday pending payment view

Phase 3 — WhatsApp Integration (Week 9)

- [ ] Meta Cloud API integration (environment config)
- [ ] Client invoice notification (on status → SENT)
- [ ] Vendor bill notification (on transaction save)
- [ ] Notification log screen
- [ ] Retry mechanism for failed notifications
- [ ] Message template submission and approval (client action)

Phase 4 — Reports & Analytics (Weeks 10–11)

- [ ] Top 3 material/cement/tool usage reports
- [ ] Site expense reports
- [ ] Revenue reports per project
- [ ] Vendor purchase reports
- [ ] Cash flow report
- [ ] PDF export for all reports

Phase 5 — Project Closure Report (Week 12)

- [ ] Closure report aggregation logic
- [ ] PDF builder (labour + materials + finance + timeline)
- [ ] R2 upload and retrieval
- [ ] Project close flow (status lock + report trigger)

Phase 6 — Extra Work & Directory (Week 13)

- [ ] Extra work log (per project)
- [ ] Unbilled summary across all projects
- [ ] Full vendor/shop/electrician directory screens
- [ ] WhatsApp notification history per contact

Phase 7 — Backup, Export & Polish (Week 13–14)

- [ ] Automated daily backup cron to R2
- [ ] Manual backup trigger from admin panel
- [ ] Full CSV export
- [ ] Mobile UI polish pass
- [ ] Cross-device testing
- [ ] Performance audit (slow queries, large lists)

Phase 8 — QA & Deployment (Week 14–15)

- [ ] Full feature regression testing
- [ ] Client walkthrough session
- [ ] Data seeding with initial real data (site names, workers)
- [ ] Production deployment
- [ ] Backup verification
- [ ] Handover documentation

---

12. SCALABILITY & FUTURE-PROOFING

12.1 How the App Scales to 50 Sites

- No code changes needed — schema is project-agnostic
- Indexes on project_id across all tables ensure query speed at scale
- Pagination on all list endpoints prevents large payload issues
- R2 storage scales infinitely at negligible cost

12.2 Multi-User (Phase 2)

The schema is designed to support multi-user with minimal migration:

```sql
-- Add to projects (already planned):
users
  id          UUID PK
  email       VARCHAR
  role        ENUM(ADMIN, MANAGER, VIEWER)
  created_at  TIMESTAMP

-- Add user_id FK to key tables for row-level ownership
```

NextAuth already supports multiple providers and users — only the session config and middleware need updating.

12.3 Mobile App (Phase 3)

- Next.js API routes are REST — any React Native or Flutter app can consume them without backend changes
- The same DB, same auth (token-based), same endpoints
- Only the frontend layer changes

12.4 Tally Export (Phase 2)

- A dedicated `/api/export/tally` endpoint can generate a Tally-compatible XML/CSV
- No DB changes required — export is a read-only transformation

12.5 Technology Lock-in Risks

Risk	Mitigation
Vercel pricing changes	Docker VPS option ready — same codebase
Railway DB pricing	Self-host PostgreSQL on VPS — 1 hour migration
Meta WhatsApp API changes	Notification layer abstracted — swap provider without touching modules
shadcn/ui deprecation	Components are copied into the repo — not a live dependency

---

Full App Spec v1.0 — Bismillah Construction To be updated after final feature confirmation and deployment option decision.